

Technical Specification & Database Setup

Phase 1 | Days 1 & 2: Relational Database Architecture & SQL Engines

1. Executive Summary

The goal of Days 1 and 2 was to transition the data engineering pipeline from flat-file storage (CSV) to a Relational Database Management System (RDBMS). While flat files are suitable for simple outputs, enterprise machine learning models require robust, strictly typed, and validated data. By architecting a local SQLite database and interacting with it programmatically via Python, we established the foundational layer of a production-grade data pipeline capable of enforcing data integrity before it reaches the application layer.

2. Database Architecture & Schema Design (DDL)

2.1 Data Definition Language & Schema Constraints

- **Action:** Authored `01_schema.sql` utilizing `CREATE TABLE` to architect a strictly typed `users` table.
- **Engineering Rationale:**
 - **The Problem:** CSV files lack internal validation. A pipeline can easily write the string `"five"` into a numerical price column, which will cause a neural network to crash during model training.
 - **The Solution:** An RDBMS enforces a Schema. By applying constraints such as `NOT NULL`, `UNIQUE`, and `DEFAULT`, the database physically rejects malformed or duplicate data, ensuring 100% data integrity at the storage level.

2.2 Primary Keys & Row Identification

- **Action:** Implemented `user_id INTEGER PRIMARY KEY AUTOINCREMENT` as the foundational column of the schema.
- **Engineering Rationale:** In relational architecture, every record must be absolutely distinct. The Primary Key ensures mathematical uniqueness and creates an indexed B-

Tree under the hood, allowing for lightning-fast `O(1)` query lookups even when the dataset scales to millions of rows.

3. Data Manipulation & Python Integration

3.1 Data Manipulation Language (DML)

- **Action:** Executed `INSERT INTO` statements to populate the `users` table with initial test records.
- **Engineering Rationale:** This step serves as the unit test for the DDL schema. By attempting to insert data, we verify that the database engine correctly enforces the defined data types and constraints.

3.2 Programmatic SQL Execution via Python (`sqlite3`)

- **Action:** Developed `build_db.py` utilizing Python's built-in `sqlite3` library and `pathlib` to programmatically connect to, generate, and populate `company.db`.
- **Engineering Rationale:** In production environments, Data Engineers do not manually type SQL queries into a terminal. They build automated scripts that connect to remote databases (e.g., AWS RDS, PostgreSQL). The `sqlite3.connect()` workflow perfectly mimics a production pipeline, connecting the application layer (Python) to the storage layer (SQL database) to execute scripts autonomously.

4. Command Glossary (For Quick Reference)

Command / Concept	Action Performed	System Impact
<code>CREATE TABLE</code>	Schema Generation	Allocates a new tabular structure within the database file.
<code>PRIMARY KEY</code>	Row Indexing	Enforces absolute uniqueness and creates a lookup index for the column.
<code>INSERT INTO</code>	Data Population	Writes new records into the specified table fields.

<code>sqlite3.connect()</code>	Engine Instantiation	Opens a stream to the <code>.db</code> file; creates the file if it does not exist.
<code>cursor.executescript()</code>	Batch Execution	Allows Python to run an entire <code>.sql</code> file containing multiple queries at once.
<code>connection.commit()</code>	Transaction Save	Permanently writes the pending transaction to the hard drive.
